

Guía de Conexión al Controlador Omada

mediante Open API (Authorization Code Mode)

Autor: Jaris Aizprúa B.

En colaboración con Claude AI Sonnet 5

Repositorio: github.com/jarisaizprua/omada-sdn-openapi

Tabla de Contenido

1. Introducción
2. Fundamentos del Open API de Omada
 - 2.1 Flujo del Authorization Code Mode (documentación oficial)
 - 2.2 Flujo del Client Credentials Mode (documentación oficial, no usado en esta guía)
3. Crear la Aplicación en el Controlador
4. Flujo Detallado de Autenticación (Authorization Code Mode)
 - 4.1 Login del usuario
 - 4.2 Obtener el código de autorización
 - 4.3 Obtener el access token
 - 4.4 Acceder a los recursos de la API
 - 4.5 Refrescar el access token
5. Invalidación de Tokens y Dominios del Servicio
 - 5.1 Casos en que el access token se invalida
 - 5.2 Dominios (Dedicados a CBC)
 - 5.3 Procesamiento por lotes (Batch Processing)
6. Estructura del Proyecto
7. Configuración: credentials.json
8. Script 1: omada_connect.py
9. Script 2: omada_devices.py
10. Script 3: omada_local_users.py
11. Plantilla Excel: omada_local_users_template.xlsx
12. Script 4: omada_create_local_users.py
13. Script 5: omada_delete_local_users.py
14. Referencia de Endpoints (Documentación Oficial)
 - 14.1 Get site list
 - 14.2 Get site device list
 - 14.3 Get site template list (documentado, no implementado)
 - 14.4 Get local user list
 - 14.5 Create a new local user

- 14.6 Delete an existing local user
- 15. Consideraciones de Seguridad
- 16. Instalación y Ejecución Paso a Paso
- 17. Publicación y Actualización en GitHub
- 18. Notas Finales

1. Introducción

Este documento describe, paso a paso, cómo conectarse al controlador Omada mediante su Open API usando el modo Authorization Code, y cómo utilizar los cinco scripts en Python desarrollados para: (1) autenticarse y obtener un access token, (2) consultar los dispositivos adoptados en cada site, (3) consultar los usuarios locales de hotspot configurados en cada site, (4) crear usuarios locales en uno, varios o todos los sites de forma masiva desde un Excel, y (5) eliminar usuarios locales de la misma manera.

La guía está basada en la documentación oficial del Omada Open API (OAuth 2.0) proporcionada por el fabricante, y en el desarrollo práctico realizado y probado contra un controlador real.

Repositorio de referencia: <https://github.com/jarisaizprua/omada-sdn-openapi>

2. Fundamentos del Open API de Omada

El Open API de Omada permite a los usuarios escribir aplicaciones personalizadas que interactúan con el controlador mediante servicios REST. Soporta operaciones HTTP GET y POST, y todas las respuestas se devuelven en formato JSON.

Para el acceso seguro, Omada implementa el protocolo OAuth 2.0, que permite acceder a recursos autorizados sin necesidad de exponer directamente usuario y contraseña en cada llamada. Se soportan dos modos de autorización:

- Authorization Code Mode: pensado para clientes confidenciales, con un flujo de redirección donde el usuario inicia sesión y autoriza a la aplicación cliente. Es el modo utilizado en esta guía y en los scripts.
- Client Credentials Mode: el cliente obtiene el access token directamente con su client_id y client_secret, sin necesidad de login de usuario ni código de autorización.

Nota: Cada controlador Omada Pro y cada organización tiene un límite de 10 solicitudes por segundo al Open API. Además, la mayoría de las interfaces tienen límite de concurrencia; se recomienda modificar primero las configuraciones globales antes que las de cada site.

2.1 Flujo del Authorization Code Mode (documentación oficial)

1. El usuario visita el cliente (aplicación).
2. El cliente dirige al usuario hacia el servidor de autenticación (Omada).
3. El usuario inicia sesión (/openapi/login) y autoriza al cliente de terceros.
4. El servidor de autenticación redirige al usuario a la URL de redirección definida por el cliente, adjuntando un código de autorización (redirect URL + ?code={código}).
5. El cliente usa el código de autorización para solicitar el accessToken al servidor de autenticación (Omada).
6. El servidor de autenticación (Omada) autentica al cliente y emite un access token.
7. El cliente usa el access token para solicitar recursos al servidor de recursos (Omada).

8. El servidor de recursos (Omada) confirma el access token y entrega los recursos al cliente.
9. El cliente puede renovar el access token mediante el refresh token, client ID y client secret.

2.2 Flujo del Client Credentials Mode (documentación oficial, no usado en esta guía)

1. El usuario abre el cliente, y el cliente solicita autorización al usuario.
2. El usuario acepta autorizar al cliente.
3. El cliente usa la autorización obtenida para solicitar un access token al servidor de autenticación (Omada).
4. El servidor de autenticación (Omada) autentica al cliente y emite un access token.
5. El cliente usa el access token para solicitar recursos al servidor de recursos (Omada).
6. El servidor de recursos (Omada) confirma el access token y entrega los recursos al cliente.
7. El cliente puede renovar el access token mediante el refresh token, client ID y client secret.

3. Crear la Aplicación en el Controlador

Antes de poder usar el Open API, hay que registrar una aplicación en el controlador:

1. Ingresar a Settings > Platform Integration (en la vista MSP o Global).
2. Crear una nueva aplicación, eligiendo el modo Authorization Code.
3. Definir el nombre de la aplicación y la Redirect URL (opcional; si no se define, el login se realiza directamente vía API sin redirección de navegador).
4. Una vez creada, el controlador genera un Client ID y un Client Secret, necesarios para todo el flujo de autenticación.

En la vista de detalle de la aplicación (botón 'View'), el controlador expone cuatro datos clave:

Campo en el controlador	Descripción	Uso en los scripts
Oauth Login Page Address	URL de la página de login para el flujo por navegador (Redirect URL específico).	Se guarda como referencia; no se usa en el login directo por API.
Interface Access Address	Dominio base para todas las llamadas REST (login, autorización, token, API).	Campo interface_access_address / base_url.
Omada ID	Identificador del MSP o customer en el controlador.	Campo omada_id.
Client ID	Identificador único de la aplicación registrada.	Campo client_id.

Nota: El Client Secret también se muestra en el detalle de la aplicación (a veces detrás de un ícono para revelarlo) y es obligatorio para intercambiar el código de autorización por el access token.

4. Flujo Detallado de Autenticación (Authorization Code Mode)

4.1 Login del usuario

Endpoint: POST /openapi/authorize/login

Parámetros de Query: client_id, omadac_id. Cuerpo (Body): username, password.

Ejemplo de solicitud:

```
curl "https://localhost:8043/openapi/authorize/login?client_id=<client_id>&omadac_id=<omadac_id>" \
  -H 'content-type:application/json' \
  -d '{"username":"admin","password":"tplink123"}' \
  -X POST -i -k --insecure
```

Respuesta esperada: devuelve csrfToken y sessionId, que deben guardarse para el siguiente paso.

```
{"errorCode":0,"msg":"Open API Log in successfully.","result":
{"csrfToken":"...", "sessionId":"..."}}
```

4.2 Obtener el código de autorización

Endpoint: POST /openapi/authorize/code

Parámetros de Query: client_id, response_type (siempre 'code' en este modo), omadac_id. Headers: Csrft-Token y Cookie (con el prefijo TPOMADA_SESSIONID=).

```
curl "https://localhost:8043/openapi/authorize/code?
client_id=<client_id>&omadac_id=<omadac_id>&response_type=code" \
  -H 'content-type:application/json' \
  -H 'Csrft-Token:<csrfToken>' \
  -H 'Cookie:TPOMADA_SESSIONID=<sessionId>' \
  -X POST -i -k --insecure
```

Nota: El código de autorización solo es válido durante 2 minutos; si expira, hay que repetir el proceso desde el login.

4.3 Obtener el access token

Endpoint: POST /openapi/authorize/token

Parámetros de Query: grant_type=authorization_code, code. Cuerpo: client_id, client_secret.

```
curl "https://localhost:8043/openapi/authorize/token?grant_type=authorization_code&code=<code>" \
  -H 'content-type:application/json' \
  -d '{"client_id":"<client_id>","client_secret":"<client_secret>"}' \
  -X POST -i -k --insecure
```

Respuesta: incluye accessToken, tokenType, expiresIn (segundos) y refreshToken.

4.4 Acceder a los recursos de la API

Con el access token obtenido, cada llamada a la API debe incluir el header Authorization con el prefijo AccessToken=.

```
curl "https://localhost:8043/openapi/v1/<omadaId>/sites?pageSize=1&page=1" \
-H 'content-type:application/json' \
-H 'Authorization:AccessToken=<accessToken>' \
-X GET -i -k --insecure
```

4.5 Refrescar el access token

El access token es válido 2 horas y el refresh token 14 días. Cuando el access token expira, se puede renovar sin repetir todo el flujo de login.

Endpoint: POST /openapi/authorize/token, con grant_type=refresh_token y el refresh_token vigente.

Nota: Un refresh token solo puede usarse una vez dentro de su periodo de validez; cada renovación entrega un nuevo refresh token que debe guardarse para la siguiente renovación. Si el refresh token también expira, hay que repetir el proceso de autenticación completo.

5. Invalidación de Tokens y Dominios del Servicio

5.1 Casos en que el access token se invalida

- Eliminación de la aplicación Open API.
- Eliminación del controlador.
- Restauración del controlador.
- Deshabilitar el modo MSP.
- Habilitar el modo MSP.
- Modificación de la aplicación Open API.
- Expiración natural del token.

5.2 Dominios (Dedicados a CBC)

Región	Front-end Login Domain	REST API Domain
Singapur	aps1-omada-controller.tplinkcloud.com	aps1-omada-northbound.tplinkcloud.com
Europa	euw1-omada-controller.tplinkcloud.com	euw1-omada-northbound.tplinkcloud.com
EE. UU.	use1-omada-controller.tplinkcloud.com	use1-omada-northbound.tplinkcloud.com

Para controladores locales (on-premise), tanto la dirección de login como la de acceso a la API se consultan directamente en Settings > Platform Integration > Open API, dentro del detalle de la aplicación (OAuth Login Page Address e Interface Access Address).

5.3 Procesamiento por lotes (Batch Processing)

El Open API permite procesar hasta 20 llamadas en un solo lote (Batch Processing OpenAPIs), excluyendo llamadas MSP, y todas deben usar el mismo omadaId.

En modo MSP, se pueden procesar hasta 20 llamadas MSP en lote (Batch Processing MSP OpenAPIs), usando el mismo mspId para las llamadas MSP y el customerId correspondiente para las no-MSP.

Nota: Esta guía y los scripts actuales no implementan batch processing; se documenta aquí como referencia de la especificación oficial.

6. Estructura del Proyecto

El repositorio contiene los siguientes archivos:

Archivo	Descripción
omada_connect.py	Ejecuta el flujo completo de autenticación (login, código de autorización, access token) y guarda los tokens en tokens.json.
omada_devices.py	Usa los tokens generados para recorrer automáticamente todos los sites del controlador y mostrar los dispositivos adoptados en cada uno.
omada_local_users.py	Usa los tokens generados para recorrer automáticamente todos los sites y mostrar los usuarios locales de hotspot configurados en cada uno.
omada_create_local_users.py	Lee la hoja CrearUsuarios de omada_local_users_template.xlsx y crea cada usuario en el/los site(s) indicados.
omada_delete_local_users.py	Lee la hoja EliminarUsuarios de omada_local_users_template.xlsx y elimina cada usuario del/de los site(s) indicados.
omada_local_users_template.xlsx	Plantilla de datos con las hojas CrearUsuarios, EliminarUsuarios, Log e Instrucciones.
credentials_template.json	Plantilla de referencia con placeholders (sin datos reales). Es la versión segura para subir al repositorio.
credentials.json	Archivo local con las credenciales reales del controlador. NO debe subirse al repositorio con datos reales.
tokens.json	Se genera automáticamente al ejecutar omada_connect.py. Contiene el access token y el refresh token reales; está excluido del repositorio mediante .gitignore.
requirements.txt	Lista de dependencias de Python necesarias (requests).
.gitignore	Excluye tokens.json, credentials.json y otros archivos que no deben subirse al repositorio (entornos virtuales, cachés, etc.).
README.md	Guía rápida de instalación y uso del proyecto, en el propio repositorio.
Guia_Omada_OpenAPI.pdf	Esta guía detallada, publicada en el repositorio en formato PDF.

Nota: credentials_template.json se agregó como referencia para consultar el formato esperado del archivo de credenciales sin arriesgarse a sobrescribir accidentalmente el credentials.json local con datos reales.

7. Configuración: credentials.json

Antes de ejecutar cualquier script, se debe crear (o editar) el archivo credentials.json en la raíz del proyecto, con la siguiente estructura (igual a la de credentials_template.json):

```
{
  "oauth_login_page_address": "https://YOUR_OAUTH_LOGIN_PAGE_ADDRESS",
  "interface_access_address": "https://YOUR_INTERFACE_ACCESS_ADDRESS",
  "omada_id": "YOUR_OMADA_ID",
  "client_id": "YOUR_CLIENT_ID",
  "client_secret": "YOUR_CLIENT_SECRET",
  "username": "YOUR_USERNAME",
  "password": "YOUR_PASSWORD"
}
```

Campo	Origen en el controlador
oauth_login_page_address	Settings > Platform Integration > Open API > View > OAuth Login Page Address
interface_access_address	Settings > Platform Integration > Open API > View > Interface Access Address
omada_id	Settings > Platform Integration > Open API > View > Omada ID
client_id	Settings > Platform Integration > Open API > View > Client ID
client_secret	Settings > Platform Integration > Open API > View (detalle de la aplicación)
username / password	Credenciales del usuario del controlador con permisos de vista sobre sites, dispositivos y hotspot

8. Script 1: omada_connect.py

Este script ejecuta el flujo completo de autenticación en modo Authorization Code y deja los tokens listos para ser usados por omada_devices.py y omada_local_users.py. A continuación se explica cada función:

load_credentials(path)

Lee y valida la existencia del archivo credentials.json, devolviendo su contenido como diccionario de Python.

mask(value, visible=4)

Enmascara valores sensibles (contraseñas, tokens, secretos) dejando visibles solo los últimos N caracteres. Con visible=0 oculta el valor por completo. Se usa en toda la salida por consola para no exponer datos reales.

login(base_url, client_id, omada_id, username, password)

Realiza POST /openapi/authorize/login. Envía client_id y omada_id como parámetros de query, y username/password en el cuerpo. Devuelve csrfToken y sessionId.

get_authorization_code(base_url, client_id, omada_id, csrf_token, session_id)

Realiza POST /openapi/authorize/code usando los headers Csrf-Token y Cookie obtenidos del login. Devuelve el código de autorización (válido 2 minutos).

get_access_token(base_url, client_id, client_secret, auth_code)

Realiza POST /openapi/authorize/token con grant_type=authorization_code, intercambiando el código de autorización por el access token, token type, tiempo de expiración y refresh token.

refresh_access_token(base_url, client_id, client_secret, refresh_token)

Realiza POST /openapi/authorize/token con grant_type=refresh_token, para renovar el access token sin repetir el login completo. Disponible como función de apoyo para uso futuro.

save_tokens(tokens, omada_id, base_url, oauth_login_page_address, path)

Guarda accessToken, refreshToken, tokenType, expiresIn y los datos de conexión en tokens.json, para que los demás scripts puedan reutilizarlos sin pedir credenciales de nuevo.

main()

Orquesta el flujo completo: carga credenciales, hace login, obtiene el código de autorización, obtiene el access token y guarda los tokens.

Ejecución:

```
python omada_connect.py
```

9. Script 2: omada_devices.py

Este script consume los tokens generados por omada_connect.py para recorrer automáticamente todos los sites del controlador y mostrar, para cada uno, los dispositivos adoptados: nombre, modelo, firmware, IP, estado y MAC. No requiere selección manual de site: si el controlador tiene varios sites, se consultan y muestran todos, uno tras otro.

load_tokens(path)

Lee tokens.json, generado previamente por omada_connect.py, y devuelve accessToken, omada_id e interface_access_address.

mask(value, visible=4)

Misma función de enmascaramiento que en omada_connect.py, usada aquí para ocultar parcialmente el access token y las direcciones MAC en la salida.

get_sites(base_url, omada_id, access_token, page, page_size)

Realiza GET /openapi/v1/{omadaId}/sites para obtener la lista completa de sites disponibles en el controlador.

get_devices(base_url, omada_id, site_id, access_token, page, page_size)

Realiza GET /openapi/v1/{omadaId}/sites/{siteId}/devices, devolviendo la lista de dispositivos de un site y el total de registros.

print_devices(site_name, devices)

Imprime en consola, para un site dado, una tabla con nombre, modelo, firmware, IP, estado (traducido a texto) y MAC (parcialmente enmascarada) de cada dispositivo. Usa columnas de ancho fijo separadas por un espaciador constante, para que la alineación no se rompa aunque el firmware sea muy largo.

main()

Orquesta el flujo: carga los tokens, obtiene todos los sites y, para cada uno, obtiene y muestra sus dispositivos.

Ejecución (requiere haber ejecutado antes omada_connect.py):

```
python omada_devices.py
```

Salida esperada (ejemplo, con MAC enmascarada):

Usando accessToken: *****3vZX

Site: SITE_LOCAL_USERS (6aa8...) - 0 dispositivo(s)
No se encontraron dispositivos en este site.

```
Site: SITE_PRUEBAS (6978...) - 2 dispositivo(s)
  Nombre      Modelo      Firmware      IP              Estado
  -----
  SWITCH_POE  ES205GP      1.0.3 Build... 192.168.68.117  Disconnected
  EAP_AX3000  EAP655-Wal   1.4.3 Build... 192.168.68.105  Disconnected
```

10. Script 3: omada_local_users.py

Este script consume los tokens generados por omada_connect.py para recorrer automáticamente todos los sites del controlador y mostrar, para cada uno, los usuarios locales de hotspot configurados: usuario, nombre, teléfono, estado, tipo de binding MAC, MAC y fecha de expiración.

Nota: La API devuelve también un campo password para cada usuario local. Este script no lo imprime en consola en ningún momento, para evitar exponer contraseñas de usuarios de hotspot en texto plano.

load_tokens(path)

Misma función que en omada_devices.py: lee tokens.json y devuelve accessToken, omada_id e interface_access_address.

mask(value, visible=4)

Misma función de enmascaramiento usada en los otros scripts, aplicada aquí al access token y a las direcciones MAC de los usuarios locales.

get_sites(base_url, omada_id, access_token, page, page_size)

Realiza GET /openapi/v1/{omadaId}/sites para obtener la lista completa de sites.

get_local_users(base_url, omada_id, site_id, access_token, page, page_size)

Realiza GET /openapi/v1/{omadaId}/sites/{siteId}/hotspot/localusers, devolviendo los usuarios locales de un site y el total de registros. Maneja el código de error -33000 (site inexistente) documentado para este endpoint.

format_expiration(expiration_ms)

Convierte el campo expirationTime (en milisegundos) a una fecha legible (AAAA-MM-DD HH:MM), o devuelve '-' si el usuario no tiene expiración configurada.

print_local_users(site_name, users)

Imprime en consola, para un site dado, una tabla con usuario, nombre, teléfono, estado, tipo de binding, MAC (enmascarada) y fecha de expiración de cada usuario local.

main()

Orquesta el flujo: carga los tokens, obtiene todos los sites y, para cada uno, obtiene y muestra sus usuarios locales.

Ejecución (requiere haber ejecutado antes omada_connect.py):

```
python omada_local_users.py
```

Salida esperada (ejemplo, con MAC enmascarada):

Usando accessToken: *****3vZX

```
Site: SITE_LOCAL_USERS (6aa8...) - 3 usuario(s)
  Usuario  Nombre  Teléfono  Habilitado  Binding  Expira
-----
  usuario_3                True      No binding  2026-12-31 23:59
  usuario_2                True      No binding  2026-12-31 23:59
```

usuario_1	True	No binding	2026-12-31 23:59
-----------	------	------------	------------------

Site: SITE_PRUEBAS (6978...) - 0 usuario(s)

No hay usuarios locales configurados en este site.

11. Plantilla Excel: omada_local_users_template.xlsx

Para crear y eliminar usuarios locales de forma masiva, sin escribir los datos directamente en el código, se usa un archivo Excel como fuente de datos y como bitácora de resultados. El archivo tiene cuatro hojas:

Hoja	Descripción
CrearUsuarios	Una fila por cada usuario local a crear, con site_name y todos los campos de CreateLocalUserOpenApiVO (ver sección 14.5). La fila 2 viene como ejemplo, en gris cursiva; debe borrarse o sobrescribirse antes de usar el archivo en serio.
EliminarUsuarios	Una fila por cada usuario local a eliminar, identificado por site_name + userName.
Log	Se actualiza automáticamente en cada ejecución de omada_create_local_users.py y omada_delete_local_users.py. No se edita a mano.
Instrucciones	Explica el propósito de cada hoja y el significado de cada columna de CrearUsuarios (códigos de bindingType, authTimeout, trafficLimitFrequency, etc.).

11.1 Formatos admitidos en la columna site_name

Tanto en CrearUsuarios como en EliminarUsuarios, la columna site_name admite tres formatos, resueltos por la función resolve_target_sites() (presente en ambos scripts):

- Un solo site: SITE_A
- Varios sites separados por coma: SITE_A, SITE_B — la operación se repite en cada uno.
- Todos los sites del controlador: la palabra clave TODOS (también se aceptan ALL o *).

Nota: Cada combinación usuario + site se procesa y registra por separado en la hoja Log. Si un nombre de site no existe, esa combinación específica queda como ERROR sin detener el procesamiento de las demás filas ni de los demás sites.

11.2 Hoja Log

Cada fila de la hoja Log representa una operación individual (una combinación usuario + site), con las siguientes columnas: timestamp, script, accion (crear/eliminar), site_name, userName, resultado (OK/ERROR) y mensaje (texto devuelto por la API o descripción del error).

12. Script 4: omada_create_local_users.py

Este script lee la hoja CrearUsuarios del Excel, resuelve el o los sites destino de cada fila, arma el cuerpo JSON esperado por la API y crea cada usuario, registrando el resultado en la hoja Log.

load_tokens(path)

Misma función que en los demás scripts: lee tokens.json y devuelve accessToken, omada_id e interface_access_address.

mask(value, visible=4)

Misma función de enmascaramiento usada en el resto del proyecto.

get_sites(base_url, omada_id, access_token, page, page_size)

Obtiene la lista completa de sites del controlador (GET /openapi/v1/{omadaId}/sites).

build_site_map(sites)

Construye un diccionario {nombre_site: siteld} para resolver nombres a IDs.

resolve_target_sites(site_field, site_map)

Interpreta el contenido de la celda site_name (un site, varios separados por coma, o TODOS/ALL/*) y devuelve la lista de (nombre_site, siteld) a procesar.

read_create_rows(path, sheet)

Lee todas las filas con datos de la hoja CrearUsuarios y las devuelve como una lista de diccionarios (columna -> valor).

to_bool(value, default)

Convierte el contenido de una celda (TRUE/FALSE, 1/0, texto) a booleano de Python.

to_int(value, default)

Convierte el contenido de una celda a entero, aplicando un valor por defecto si está vacía.

build_payload(row)

Arma el cuerpo JSON (CreateLocalUserOpenApiVO) esperado por la API a partir de una fila del Excel, incluyendo el cálculo de expirationTime en milisegundos a partir de expiration_days.

create_local_user(base_url, omada_id, site_id, access_token, payload)

Realiza POST /openapi/v1/{omadaId}/sites/{siteld}/hotspot/localusers con el payload armado.

append_log(path, site_name, user_name, resultado, mensaje, sheet)

Agrega una fila a la hoja Log del Excel, con fecha y hora, y guarda el archivo.

main()

Orquesta el flujo: carga tokens y sites, lee las filas de CrearUsuarios y, para cada una, resuelve los sites destino, arma el payload una sola vez, y lo envía a cada site, registrando cada resultado por separado.

Nota: El script hace una pausa de 0.2 segundos entre cada solicitud, para no acercarse al límite de 10 solicitudes por segundo del controlador, mencionado en la sección 2.

Ejecución (requiere haber ejecutado antes `omada_connect.py` y haber editado la hoja `CrearUsuarios`):

```
python omada_create_local_users.py
```

13. Script 5: omada_delete_local_users.py

Este script lee la hoja EliminarUsuarios del Excel, resuelve el o los sites destino de cada fila, busca el id de cada usuario por su userName (la API de eliminación requiere el id, no el userName) y lo elimina, registrando el resultado en la hoja Log.

load_tokens(path)

Misma función que en los demás scripts.

mask(value, visible=4)

Misma función de enmascaramiento.

get_sites / build_site_map / resolve_target_sites

Mismas funciones que en omada_create_local_users.py, para resolver site_name a uno o más siteId.

get_local_users(base_url, omada_id, site_id, access_token, page, page_size)

Obtiene la lista de usuarios locales de un site (GET .../hotspot/localusers), usada para encontrar el id del usuario a eliminar.

find_user_id(users, user_name)

Busca, dentro de la lista de usuarios de un site, el que coincide con userName, y devuelve su id.

delete_local_user(base_url, omada_id, site_id, access_token, user_id)

Realiza DELETE /openapi/v1/{omadaId}/sites/{siteId}/hotspot/localusers/{id}.

read_delete_rows(path, sheet)

Lee todas las filas con datos de la hoja EliminarUsuarios.

append_log(path, site_name, user_name, resultado, mensaje, sheet)

Misma función de registro en la hoja Log.

main()

Orquesta el flujo: carga tokens y sites, lee las filas de EliminarUsuarios y, para cada una, resuelve los sites destino, busca el usuario en cada uno (cacheando la lista de usuarios por site para no repetir la consulta) y lo elimina.

Nota: La lista de usuarios de cada site se consulta una sola vez y se reutiliza (cache en memoria) mientras dure la ejecución del script, para reducir el número de llamadas a la API cuando se eliminan varios usuarios del mismo site.

Ejecución (requiere haber ejecutado antes omada_connect.py y haber editado la hoja EliminarUsuarios):

```
python omada_delete_local_users.py
```

14. Referencia de Endpoints (Documentación Oficial)

14.1 Get site list

GET /openapi/v1/{omadaId}/sites — Devuelve la lista de sites del controlador. Usado por `get_sites()` en `omada_devices.py` y `omada_local_users.py`.

Campo de respuesta	Descripción
siteId	Identificador único del site.
name	Nombre del site.
region	Región configurada para el site.
timeZone	Zona horaria del site.
scenario	Escenario de uso configurado (por ejemplo, Hotel).
type	Tipo de site (0: basic, 1: pro).

14.2 Get site device list

GET /openapi/v1/{omadaId}/sites/{siteId}/devices — Devuelve la lista de dispositivos adoptados en un site. Requiere el permiso Site Device Manager View Only o MSP Device Manager View Only. Usado por `get_devices()` en `omada_devices.py`.

Parámetros de la solicitud

Parámetro	Ubicación	Requerido	Descripción
omadaId	path	Sí	Omada ID.
siteId	path	Sí	ID del site.
page	query	Sí	Número de página, inicia en 1.
pageSize	query	Sí	Registros por página (1-1000).
sorts.name / sorts.status / sorts.ip	query	No	Orden ascendente (asc) o descendente (desc) por el campo correspondiente.
searchKey	query	No	Búsqueda difusa por nombre, MAC o IP.
filters.tag	query	No	Filtro por nombre de tag.

Campos de respuesta (DeviceInfo)

Campo	Descripción
mac	Dirección MAC del dispositivo.
name	Nombre del dispositivo.
type	Tipo de dispositivo.
subtype	Subtipo (solo switches): smart = serie no ágil; es = serie ágil.
deviceSeriesType	0: básico, 1: pro.

Campo	Descripción
model	Modelo del dispositivo con versión.
modelName	Nombre del modelo.
ip	Dirección IP del dispositivo.
ipv6	Lista de direcciones IPv6.
uptime	Tiempo de actividad del dispositivo.
status	Estado general (ver tabla 14.2.1).
detailStatus	Estado detallado (ver tabla 11.2.2).
modelVersion	Versión del modelo, por ejemplo 3.0.
lastSeen	Marca de tiempo de la última vez visto.
cpuUtil	Uso de CPU (%).
memUtil	Uso de memoria (%).
sn	Número de serie del dispositivo.
licenseStatus	Estado de licencia (solo cloud): 0 inactiva, 1 sin vincular, 2 expirada, 3 activa.
tagName	Nombre del tag asignado.
uplinkDeviceMac / uplinkDeviceName / uplinkDevicePort	Datos del dispositivo y puerto de uplink.
linkSpeed	Velocidad del puerto uplink: 0 Auto, 1 10M, 2 100M, 3 1000M, 4 2500M, 5 10G, 6 5G, 7 25G, 8 100G.
duplex	Modo dúplex del puerto uplink: 0 Auto, 1 Half, 2 Full.
switchConsistent	Indica si el dispositivo puede ser adoptado por el site.
publicIp	IP pública del dispositivo.
firmwareVersion	Versión de firmware.
compatible	Tipo de compatibilidad del dispositivo.
active	Indica si el dispositivo está activado.
inWhitelList	Indica si el dispositivo está en la lista blanca.
supportAfc	Indica si el dispositivo soporta AFC.

14.2.1 Códigos de status (general)

Código	Estado
0	Disconnected
1	Connected
2	Pending
3	Heartbeat Missed
4	Isolated

Estos son los códigos que el script `omada_devices.py` traduce a texto en la función `print_devices()`.

14.2.2 Códigos de detailStatus (referencia completa)

Código	Estado detallado
0	Disconnected
1	Disconnected (Migrating)
10	Provisioning
11	Configuring
12	Upgrading
13	Rebooting
14	Connected
15	Connected (Wireless)
16	Connected (Migrating)
17	Connected (Wireless, Migrating)
20	Pending
21	Pending (Wireless)
22	Adopting
23	Adopting (Wireless)
24	Adopt Failed
25	Adopt Failed (Wireless)
26	Managed By Others
27	Managed By Others (Wireless)
30	Heartbeat Missed
31	Heartbeat Missed (Wireless)
32	Heartbeat Missed (Migrating)
33	Heartbeat Missed (Wireless, Migrating)
40	Isolated
41	Isolated (Migrating)
50	Slice Configuring

Nota: Actualmente los scripts solo traducen el campo status (tabla 14.2.1). El campo detailStatus está disponible en la respuesta cruda de la API para quien necesite mayor granularidad, y se documenta aquí como referencia oficial completa.

14.3 Get site template list (documentado, no implementado)

GET /openapi/v1/{omadaid}/sitetemplates — Devuelve la lista de plantillas de site disponibles. Requiere el permiso Global Site Template Manager View Only. Este endpoint forma parte de la documentación oficial revisada, pero no está implementado en los scripts actuales; se deja documentado como posible extensión futura.

Campo de respuesta	Descripción
id	ID de la plantilla de site.
omadaid	Omada ID.
name	Nombre de la plantilla (1 a 64 caracteres).
status	Estado de sincronización de la plantilla con los sites.
bindSiteNum	Cantidad de sites vinculados a la plantilla.
category	Categoría de la plantilla.
type	Tipo: 0 básico, 1 pro.
settings	Lista de configuraciones seleccionadas en la plantilla.

14.4 Get local user list

GET /openapi/v1/{omadaId}/sites/{siteId}/hotspot/localusers — Devuelve la lista de usuarios locales de hotspot configurados en un site. Requiere el permiso Site Hotspot Manager View Only. Usado por `get_local_users()` en `omada_local_users.py`.

Código de error específico documentado: -33000 (el site no existe), manejado explícitamente en `get_local_users()`.

Parámetros de la solicitud

Parámetro	Ubicación	Requerido	Descripción
omadaId	path	Sí	Omada ID.
siteId	path	Sí	ID del site.
page	query	Sí	Número de página, inicia en 1.
pageSize	query	Sí	Registros por página (1-1000).
sorts.name	query	No	Orden ascendente (asc) o descendente (desc) por nombre.
searchKey	query	No	Búsqueda difusa por user_name.

Campos de respuesta (LocalUserOpenApiVO) — principales

Campo	Descripción
id	ID del usuario local.
userName	Nombre de usuario (1 a 128 caracteres).
password	Contraseña del usuario (1 a 128 caracteres). No se imprime en consola en el script.
enable	Estado habilitado/deshabilitado del usuario.
expirationTime	Fecha de expiración, en milisegundos.
bindingType	Tipo de binding MAC: 0 sin binding, 1 binding estático, 2 binding dinámico.
macAddress	Dirección MAC, solo aplica si el binding es estático o dinámico.
maxUsers	Máximo de usuarios simultáneos cuando el binding es 'sin binding' (1-2048).
name	Nombre asociado al usuario.
phone	Número de teléfono (1 a 20 caracteres).
rateLimit	Configuración de límite de velocidad (modo, perfil o límites personalizados de subida/bajada).
trafficLimitEnable / trafficLimit / trafficUsed / trafficLeft	Configuración y consumo del límite de tráfico (en MB).
trafficLimitFrequency	Frecuencia del límite de tráfico: 0 total, 1 diario, 2 semanal, 3 mensual.
used	Cantidad de usos registrados.
overtime	Indica si ya se superó la expirationTime.
portals	Lista de portales cautivos asociados.
logout	Indica si se habilita el cierre de sesión del usuario local.

Campo	Descripción
applyToAllPortals	Indica si el usuario aplica a todos los portales, incluidos los nuevos.
dailyLimitEnable / dailyLimit / dailyLimitMs / dailyLimitLeftMs	Configuración y estado del límite diario de tiempo de conexión.

Tipos de binding MAC (bindingType)

Código	Tipo de binding
0	No binding
1	Static binding
2	Dynamic binding

Nota: El campo password se documenta aquí como referencia de la especificación oficial, pero omada_local_users.py nunca lo imprime en consola por razones de privacidad de los usuarios de hotspot.

14.5 Create a new local user

POST /openapi/v1/{omadaId}/sites/{siteId}/hotspot/localusers — Crea un usuario local con los parámetros dados. Requiere el permiso Site Hotspot Manager Modify. Usado por create_local_user() en omada_create_local_users.py.

Códigos de error específicos documentados

Código	Significado
-33000	El site no existe.
-33004	Operación fallida porque se están realizando otras operaciones sobre el site (copia, restauración, sincronización de plantilla, etc.). Esperar e intentar más tarde.
-42012	Se alcanzó el límite de usuarios locales. No se pueden crear más.
-42013	El usuario local ya existe.
-42020	La dirección MAC vinculada al usuario local no es válida.
-42037	Debe seleccionarse al menos un portal antes de crear usuarios locales.
-42039	Fallo al vincular con los portales. Seleccionar portales con autenticación de usuario local habilitada.
-44111	El Grant Type no es válido.
-44112	El access token expiró. Debe reiniciarse el proceso de refreshToken.

Cuerpo de la solicitud (CreateLocalUserOpenApiVO) — campos principales

Campo	Requerido	Descripción
userName	Sí	1 a 128 caracteres ASCII visibles, sin espacios al inicio/fin ni en medio. No se puede cambiar tras la creación.
password	Sí	1 a 128 caracteres.
enable	Sí	Habilita o no el usuario.
expirationTime	Sí	Marca de tiempo de expiración, en milisegundos.
bindingType	Sí	0 sin binding, 1 binding estático, 2 binding dinámico.
macAddress	No	Solo aplica si bindingType es estático o dinámico.
maxUsers	Sí	Máximo de usuarios simultáneos cuando bindingType=0. Rango 1-2048. No se puede modificar después de inicializado.
name	No	1 a 128 caracteres, sin espacios al inicio/fin.
phone	No	1 a 20 caracteres.
rateLimit.mode	Sí	0 customRateLimit, 1 rateLimitProfileId.
rateLimit.rateLimitProfileId	No	ID de un perfil de límite de velocidad ya creado (interfaz aparte, no incluida en estos scripts).
rateLimit.customRateLimit.downLimit Enable / downLimit	Depende del modo	Habilita y define el límite de bajada, en Kbps (0-10485760).
rateLimit.customRateLimit.upLimitEnable / upLimit	Depende del modo	Habilita y define el límite de subida, en Kbps (0-10485760).

Campo	Requerido	Descripción
trafficLimitEnable / trafficLimit	trafficLimit no requerido	Habilita y define el límite de tráfico, en MB (1-10485760).
trafficLimitFrequency	No	0 total, 1 diario, 2 semanal, 3 mensual.
portals	Sí	Lista de IDs de portal a los que se vincula el usuario (obtenidos con la interfaz 'Get portal list in a site', no incluida en estos scripts).
logout	No	Habilita el cierre de sesión del usuario local.
applyToAllPortals	No	Si el usuario aplica a todos los portales, incluidos los nuevos.
dailyLimitEnable	No	Habilita el límite diario de tiempo de conexión.
dailyLimit.authTimeout	Sí (si dailyLimit se envía)	0 personalizado, 1 30 min, 2 1 hora, 3 2 horas, 4 4 horas, 5 8 horas.
dailyLimit.customTimeout	No	Usado solo si authTimeout=0: 1-1440 minutos, o 1-24 horas.
dailyLimit.customTimeoutUnit	No	1 minutos, 2 horas.

Respuesta

Devuelve un objeto CreatedResIdOpenApiVO con el id del usuario recién creado:

```
{
  "errorCode": 0,
  "msg": "",
  "result": { "id": "" }
}
```

14.6 Delete an existing local user

DELETE /openapi/v1/{omadaId}/sites/{siteId}/hotspot/localusers/{id} — Elimina un usuario local existente, identificado por su id (no por su userName). Requiere el permiso Site Hotspot Manager Modify. Usado por delete_local_user() en omada_delete_local_users.py.

Parámetros de la solicitud

Parámetro	Ubicación	Requerido	Descripción
omadaId	path	Sí	Omada ID.
siteId	path	Sí	ID del site.
id	path	Sí	ID del usuario local a eliminar.

Códigos de error específicos documentados

Código	Significado
-33000	El site no existe.
-33004	Operación fallida porque se están realizando otras operaciones sobre el site. Esperar e intentar más tarde.
-44111	El Grant Type no es válido.
-44112	El access token expiró. Debe reiniciarse el proceso de refreshToken.

Respuesta

```
{
  "errorCode": 0,
  "msg": ""
}
```

Nota: Como esta interfaz requiere el id del usuario y no su userName, omada_delete_local_users.py primero consulta la lista de usuarios locales del site (sección 14.4) para encontrar el id correspondiente antes de llamar a este endpoint.

15. Consideraciones de Seguridad

- Enmascaramiento en consola: contraseñas, `client_secret`, tokens, usuario y direcciones MAC se muestran parcial o totalmente ocultos en la salida de los cinco scripts, mediante la función `mask()`.
- El campo `password` de los usuarios locales de hotspot, devuelto por la API de usuarios locales, nunca se imprime en consola en `omada_local_users.py`.
- `omada_create_local_users.py` sí envía el `password` en texto plano en el cuerpo de la solicitud de creación (es un campo obligatorio de la API), pero nunca lo imprime en consola ni lo escribe en la hoja Log.
- Expiración de tokens: el `access token` dura 2 horas y el `refresh token` 14 días. Pasado ese tiempo hay que repetir el flujo de `omada_connect.py`.
- `tokens.json`: contiene el `access token` y el `refresh token` reales; debe excluirse siempre del control de versiones mediante `.gitignore`.
- `credentials.json` vs `credentials_template.json`: el archivo local con datos reales nunca debe subirse a un repositorio; solo la plantilla (`credentials_template.json`), con valores de ejemplo, es segura de publicar.
- `omada_local_users_template.xlsx` contiene contraseñas en texto plano en la hoja `CrearUsuarios` mientras se completa; debe tratarse como archivo sensible y no subirse a un repositorio con datos reales, igual que `credentials.json`.
- Límite de solicitudes: `omada_create_local_users.py` y `omada_delete_local_users.py` hacen una pausa de 0.2 segundos entre cada solicitud, para no acercarse al límite de 10 solicitudes por segundo del controlador.
- Verificación de certificado: los scripts usan `verify=False` en las peticiones HTTPS, ya que muchos controladores usan certificados autofirmados. En entornos productivos se recomienda usar certificados válidos y activar la verificación.

Nota: Si en algún momento un token o contraseña real queda expuesto (por ejemplo, en un log copiado o en un commit de Git), debe regenerarse el Client Secret de la aplicación en el controlador y/o cambiar la contraseña del usuario afectado de inmediato, ya que eliminar el archivo del repositorio no revoca el token por sí solo.

16. Instalación y Ejecución Paso a Paso

16.1 Requisitos

- Python 3.9 o superior.
- Un controlador Omada con Open API habilitado.
- Una aplicación creada en modo Authorization Code, con un usuario del controlador con permisos de vista sobre sites, dispositivos y hotspot.

16.2 Instalación de dependencias

```
pip install -r requirements.txt
```

16.3 Configuración

Copiar `credentials_template.json` como `credentials.json` y completarlo con los datos reales del controlador (ver sección 7).

16.4 Ejecución

1. Ejecutar el script de conexión:

```
python omada_connect.py
```

2. Verificar que se generó el archivo `tokens.json` en la misma carpeta.

3. Ejecutar el script de dispositivos (recorre todos los sites automáticamente):

```
python omada_devices.py
```

4. Ejecutar el script de usuarios locales (recorre todos los sites automáticamente):

```
python omada_local_users.py
```

5. Para crear usuarios en masa: editar la hoja `CrearUsuarios` de `omada_local_users_template.xlsx` y ejecutar:

```
python omada_create_local_users.py
```

6. Para eliminar usuarios en masa: editar la hoja `EliminarUsuarios` de `omada_local_users_template.xlsx` y ejecutar:

```
python omada_delete_local_users.py
```

7. Revisar la hoja Log del Excel para confirmar el resultado de cada operación de creación o eliminación.

17. Publicación y Actualización en GitHub

Repositorio: <https://github.com/jarisaizprua/omada-sdn-openapi>

17.1 Primera publicación

```
git init
git add .
git commit -m "Initial commit: Omada OpenAPI connection and device scripts"
git branch -M main
git remote add origin https://github.com/jarisaizprua/omada-sdn-openapi.git
git push -u origin main
```

Nota: Antes de este paso, verificar con 'git status' que tokens.json y credentials.json no aparezcan listados para el commit, y que solo credentials_template.json (con placeholders) esté incluido. Verificar también que omada_local_users_template.xlsx no contenga contraseñas ni datos reales en la hoja CrearUsuarios antes de subirlo; de lo contrario, excluirlo también en .gitignore o limpiar la hoja antes del commit.

17.2 Actualizar el repositorio (amend) tras un cambio local

Cuando el repositorio tiene un solo commit y se hace un ajuste (por ejemplo, corregir el .gitignore, editar el README.md, o actualizar esta guía), es preferible reescribir ese commit en vez de sumar uno nuevo:

```
git status
git add <archivo modificado>
git commit --amend -m "Initial commit: Omada OpenAPI connection and device scripts"
git push --force origin main
```

Nota: git push --force sobrescribe el historial remoto; usarlo con cuidado y solo cuando se está seguro de que nadie más ha basado trabajo sobre ese commit.

17.3 Caso real documentado: .gitignore sin el punto

Durante el desarrollo de este proyecto, el archivo .gitignore se guardó inicialmente como 'gitignore' (sin el punto inicial), por lo que Git no lo reconoció como archivo de configuración y tokens.json terminó subido al repositorio público con tokens reales.

Pasos aplicados para corregirlo:

```
mv gitignore .gitignore
git status
git rm --cached tokens.json
git add .gitignore
git commit --amend -m "Initial commit: Omada OpenAPI connection and device scripts"
git push --force origin main
```

Nota: Adicionalmente, se regeneró el Client Secret de la aplicación en el controlador para invalidar el access token y refresh token que habían quedado expuestos, ya que eliminarlos del repositorio no los revoca.

17.4 Esta guía se publica en PDF, no en Word

El .gitignore del proyecto excluye el archivo .docx de esta guía; en su lugar, el repositorio publica la versión exportada en PDF (Guia_Omada_OpenAPI.pdf), que es la que debe actualizarse en cada cambio relevante del proyecto.

18. Notas Finales

- Esta guía cubre el modo Authorization Code, que es el implementado en los scripts. El modo Client Credentials queda documentado en la sección 2.2 como referencia, pero no está implementado.
- El endpoint de plantillas de site (sitetemplates) está documentado en la sección 14.3 como posible extensión futura del proyecto.
- Tanto omada_devices.py como omada_local_users.py recorren automáticamente todos los sites del controlador, sin requerir selección manual.
- omada_create_local_users.py y omada_delete_local_users.py admiten operar sobre un solo site, varios sites separados por coma, o todos los sites del controlador con la palabra clave TODOS, tanto para crear como para eliminar usuarios locales.
- La interfaz 'Get portal list in a site', necesaria para obtener los IDs de portal que exige el campo portals al crear un usuario, no está implementada en este proyecto; los IDs deben obtenerse por otro medio y cargarse manualmente en la hoja CrearUsuarios.
- Se recomienda revisar periódicamente los permisos del usuario y la aplicación Open API en el controlador, y rotar el Client Secret si se sospecha alguna exposición.